



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 35

Dr. Asad Mahmood

April 27, 2026

Lecture Outline

- Announcements (Assignment and Quiz 4)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.7 Pipelined Datapath and Control
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.7 Pipelined Datapath and Control

Chapter 4

The Processor

Chapter Outline

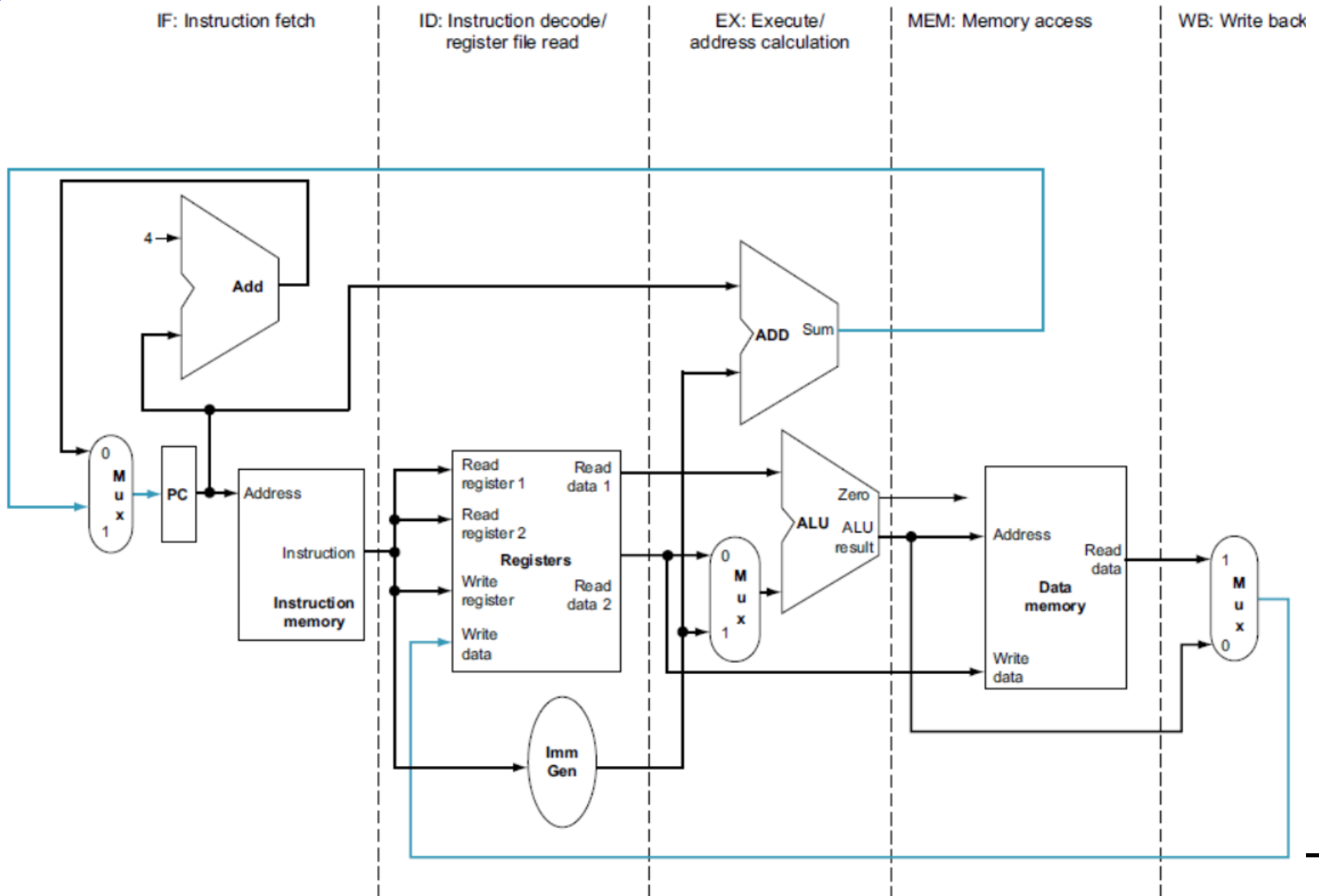
Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

RISC-V Pipelined Datapath

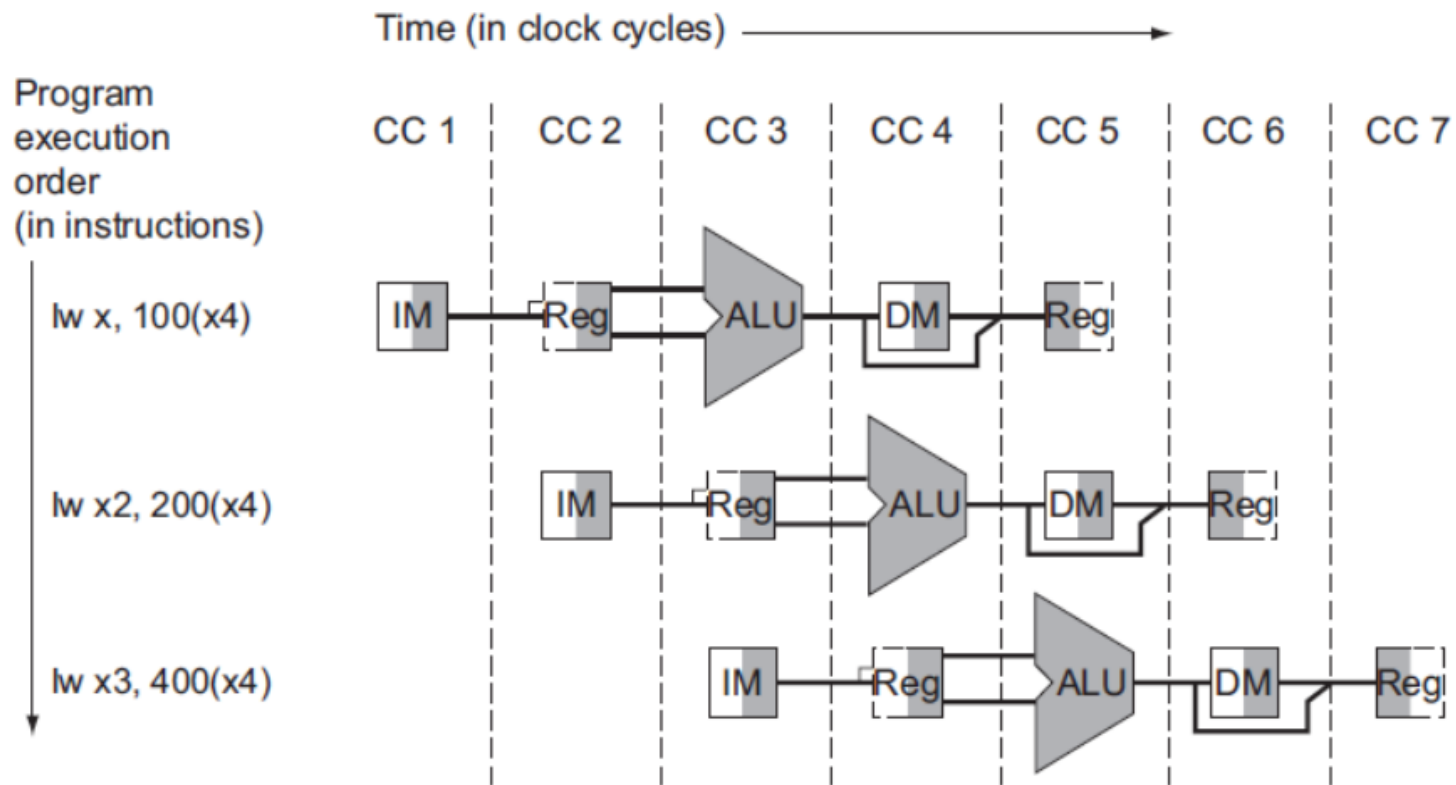
- We saw previously that instruction execution can be divided into 5 stages of pipeline
- How many instructions in parallel?
- RISC-V datapath can correspondingly be divided into 5 pieces/segments
- Data moves left-to-right in the datapath with two exceptions

RISC-V Single-cycle Datapath



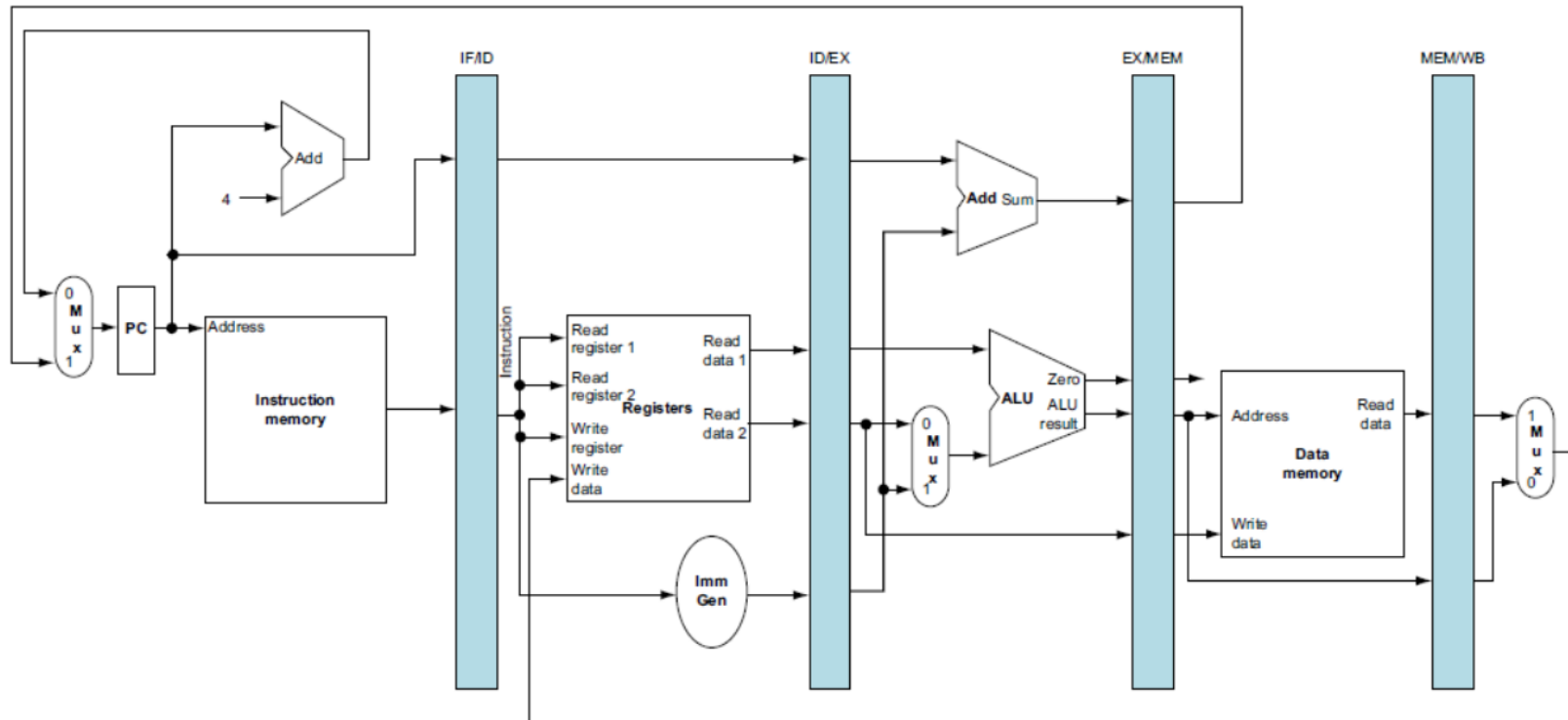
RISC-V Single-cycle Pipelined Datapath

- For ease of visualization, assume each instruction has its own dedicated datapath



Pipeline registers

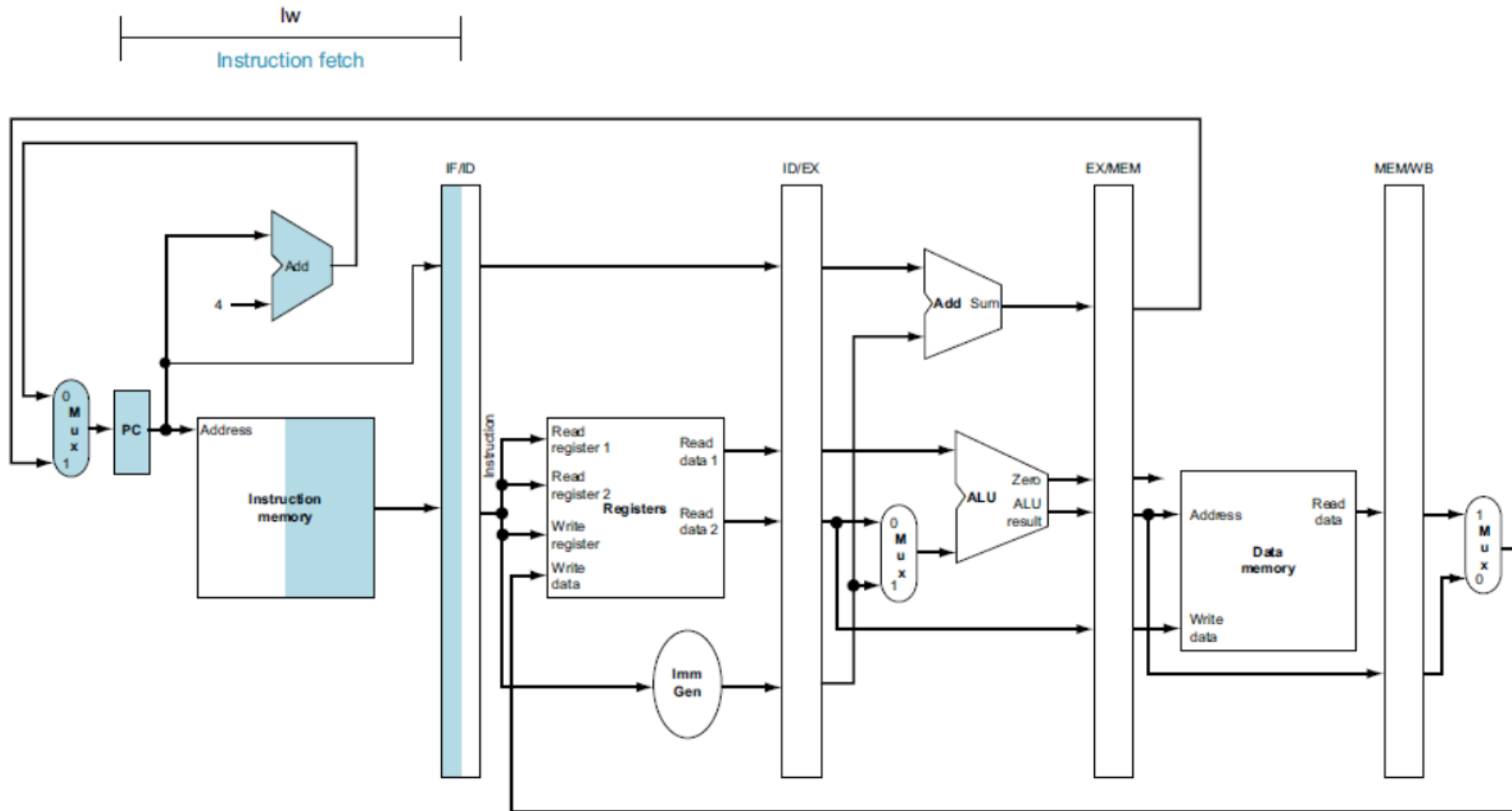
- In reality, a single datapath gets shared amongst instructions via use of registers
 - No pipeline register after the WB stage
 - Registers width?



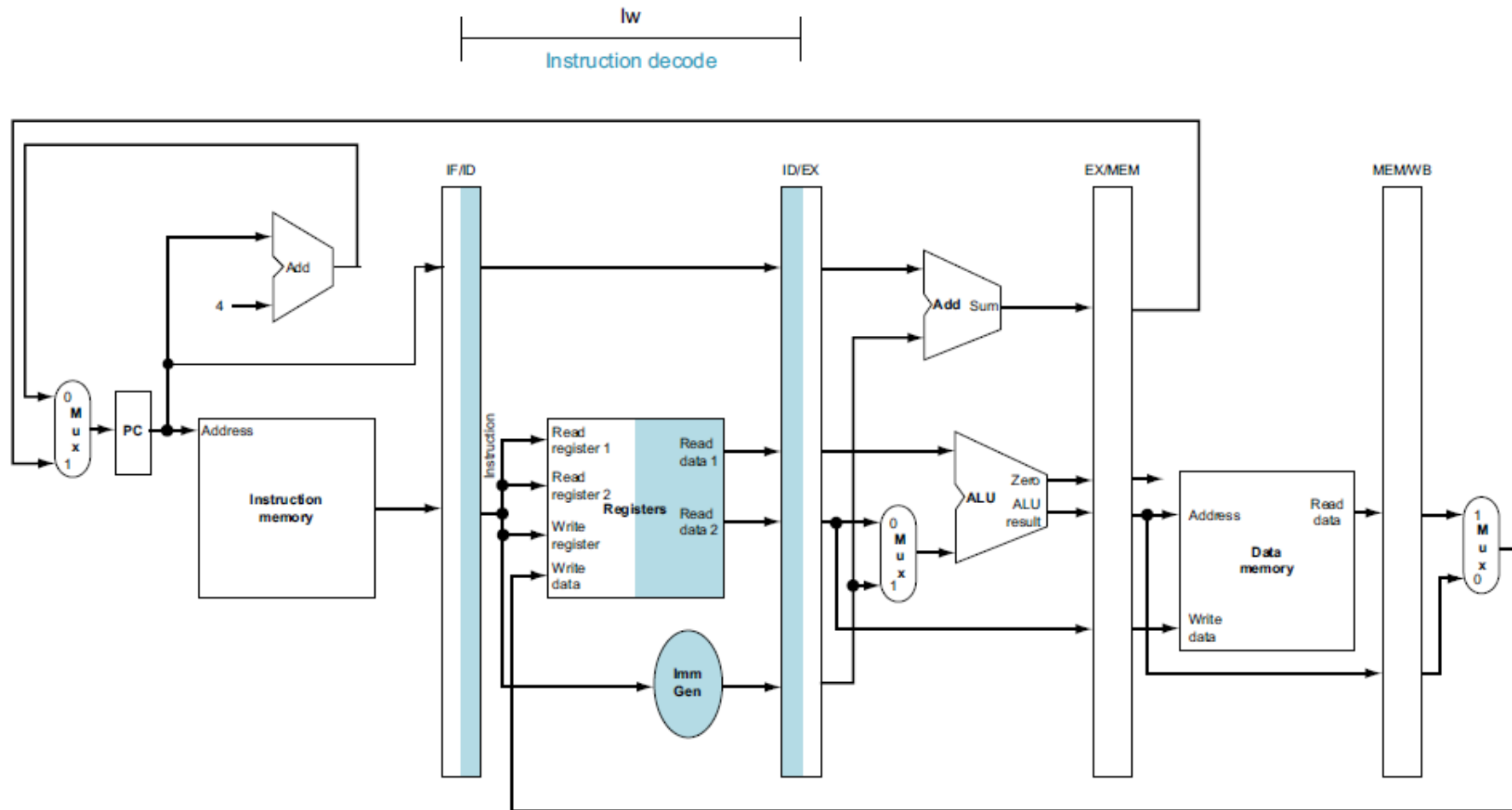
Pipeline Operation

- Pipeline operation will be demonstrated via cycle-by-cycle flow of instructions through the pipelined datapath. This will be done using two types of figures:
 1. “Single-clock-cycle” pipeline figures
 - Shows pipeline usage in a single cycle
 - Highlight resources used
 2. “Multi-clock-cycle” figures
 - Graph of operation over multiple clock cycles
- We will first look at “single-clock-cycle” diagrams for load & store

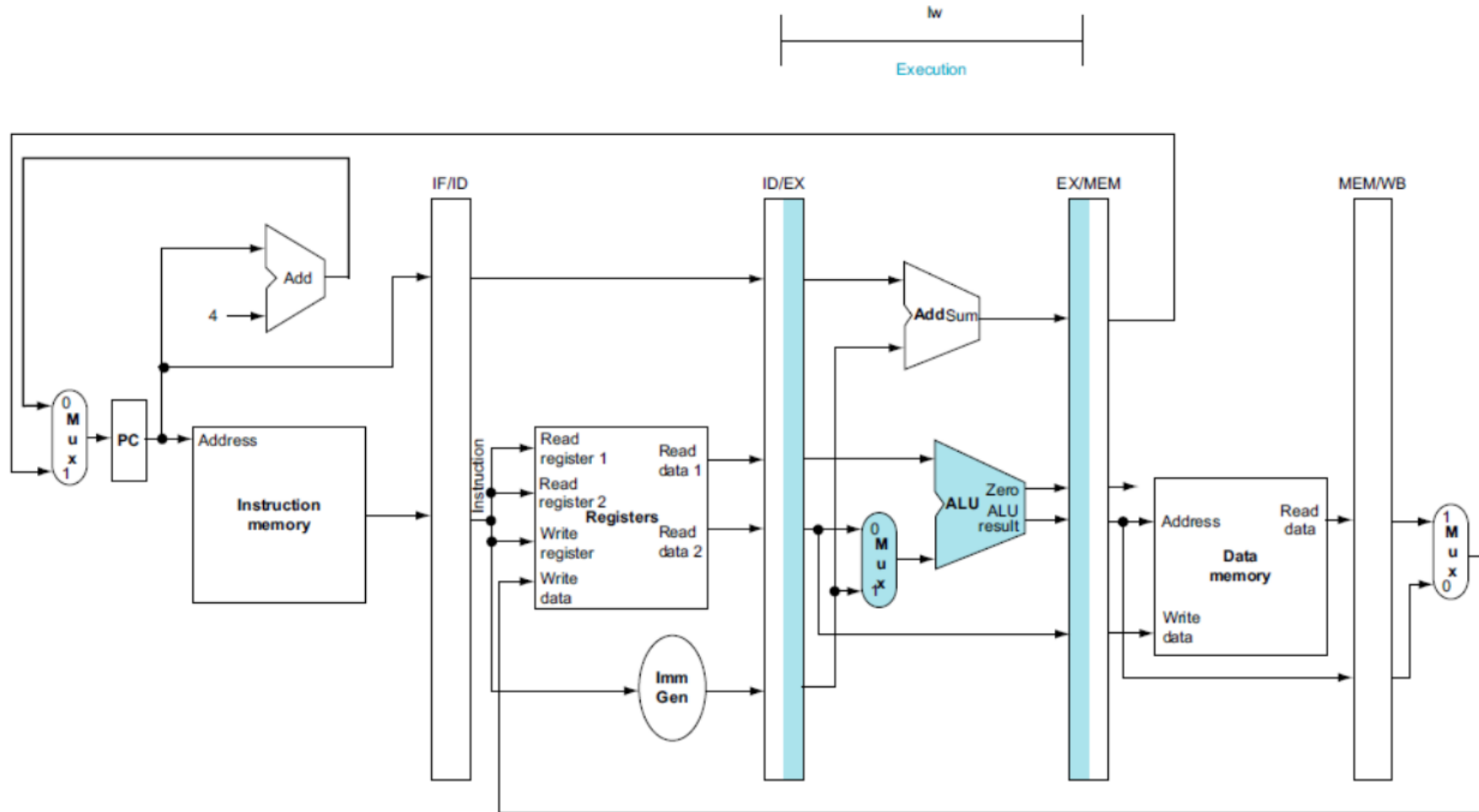
IF for Load, Store, ...



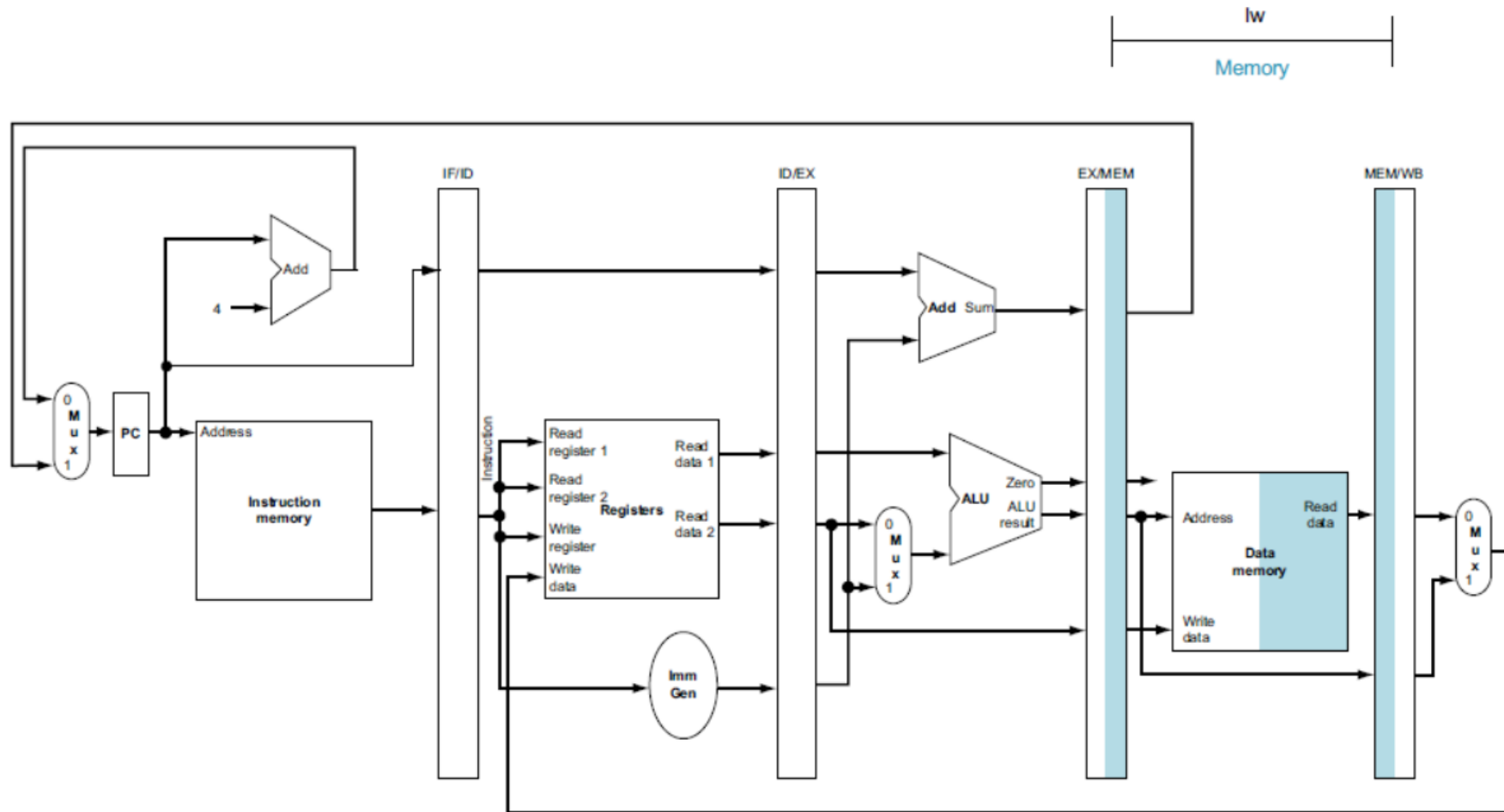
ID for Load, Store, ...



EX for Load

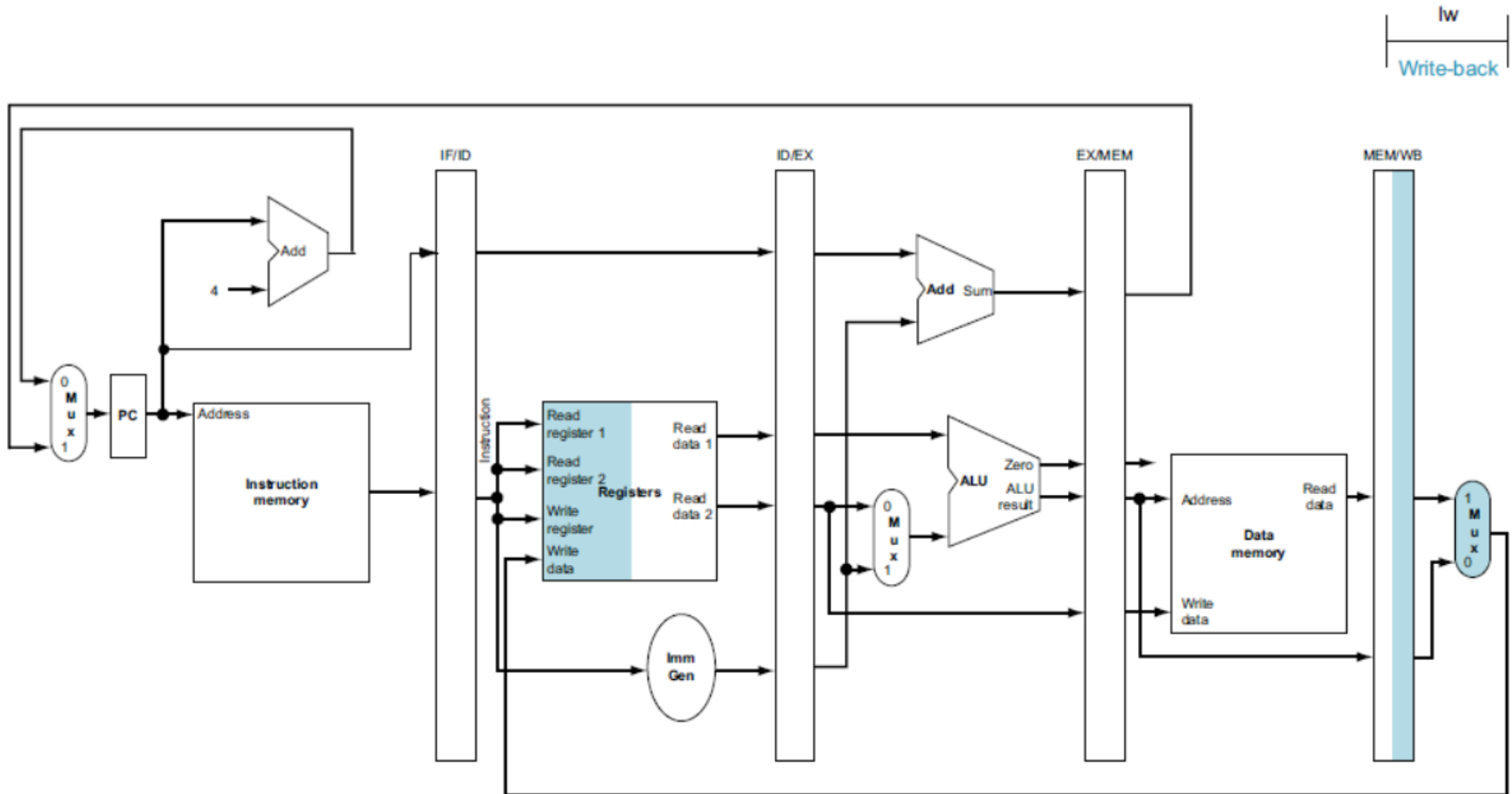


MEM for Load

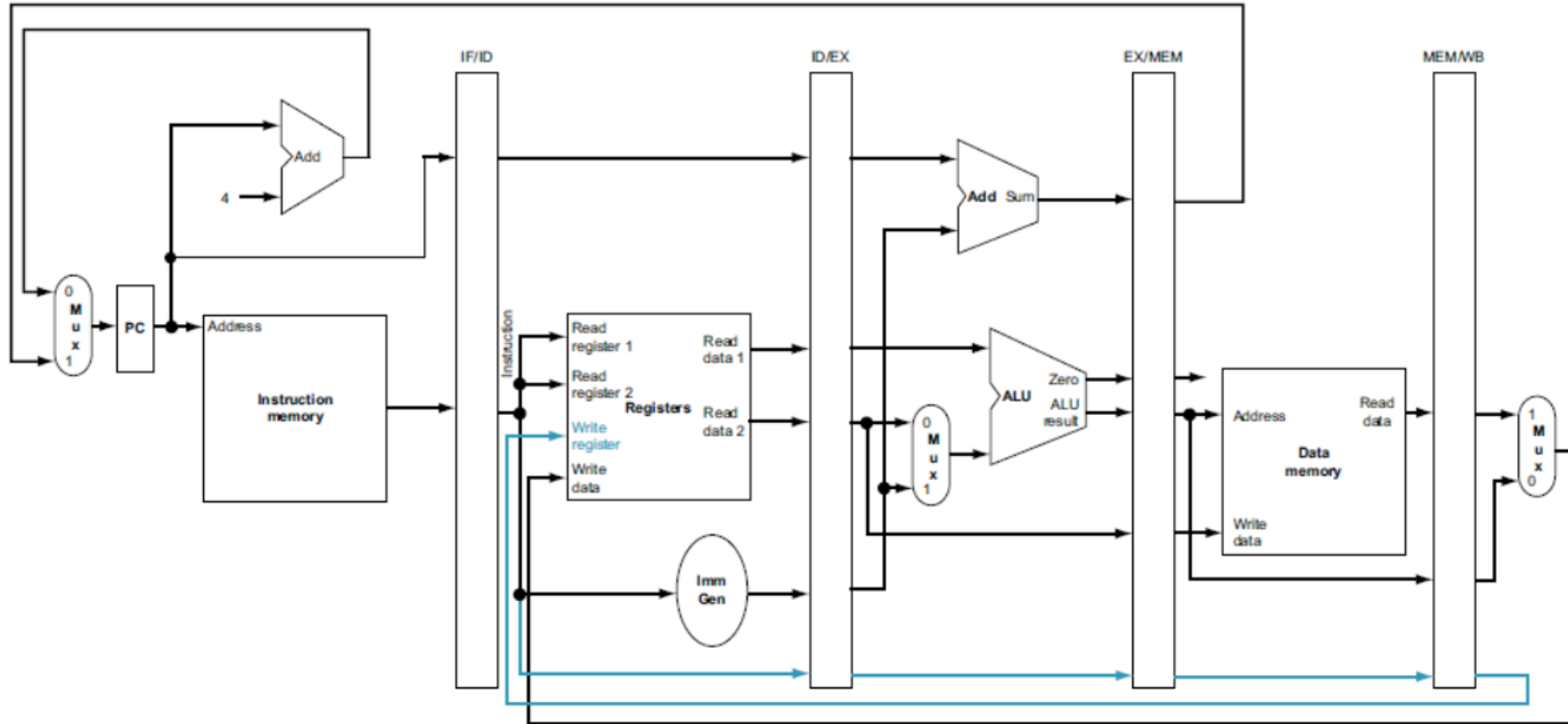


WB for Load

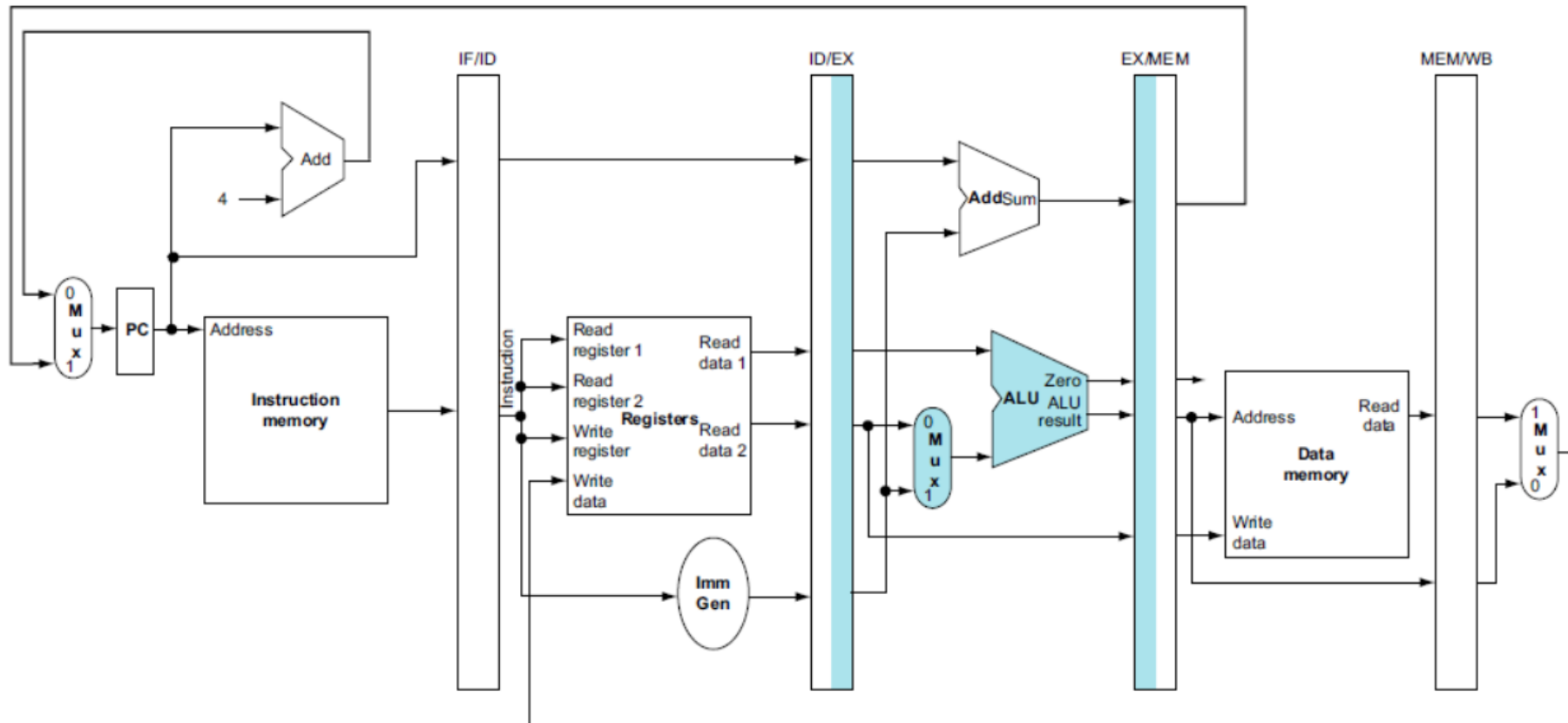
A bug in this figure!



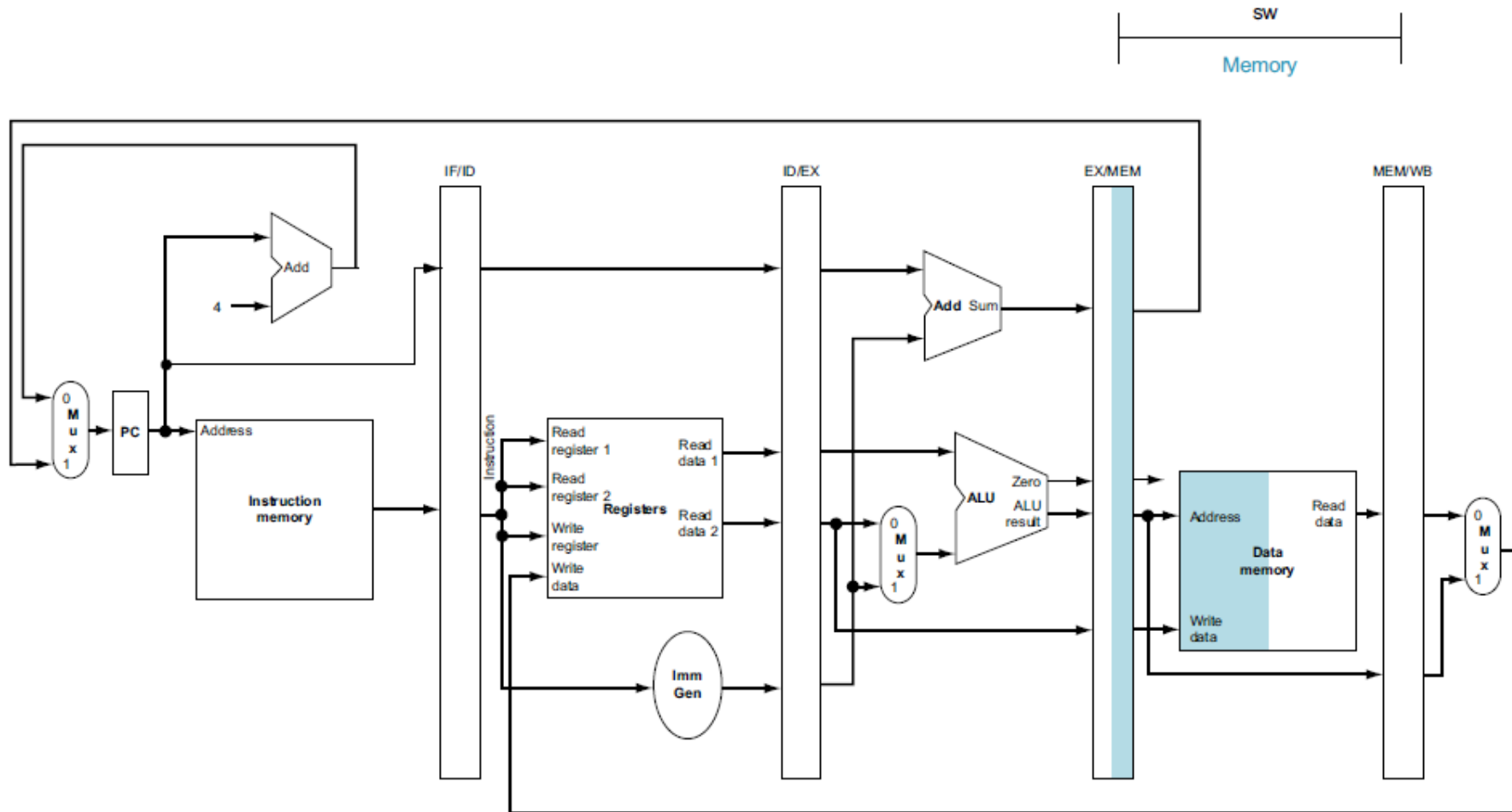
Corrected Datapath for Load



EX for Store

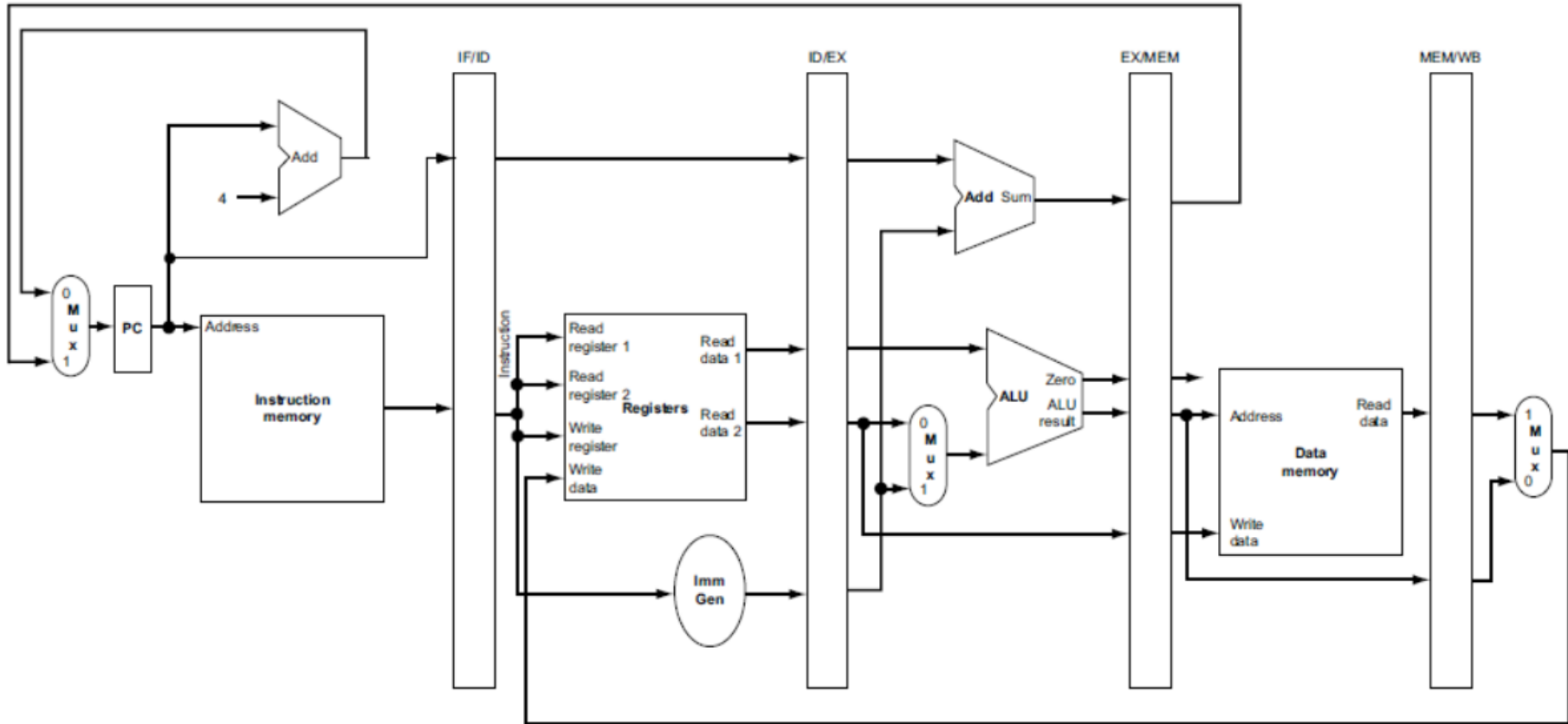


MEM for Store

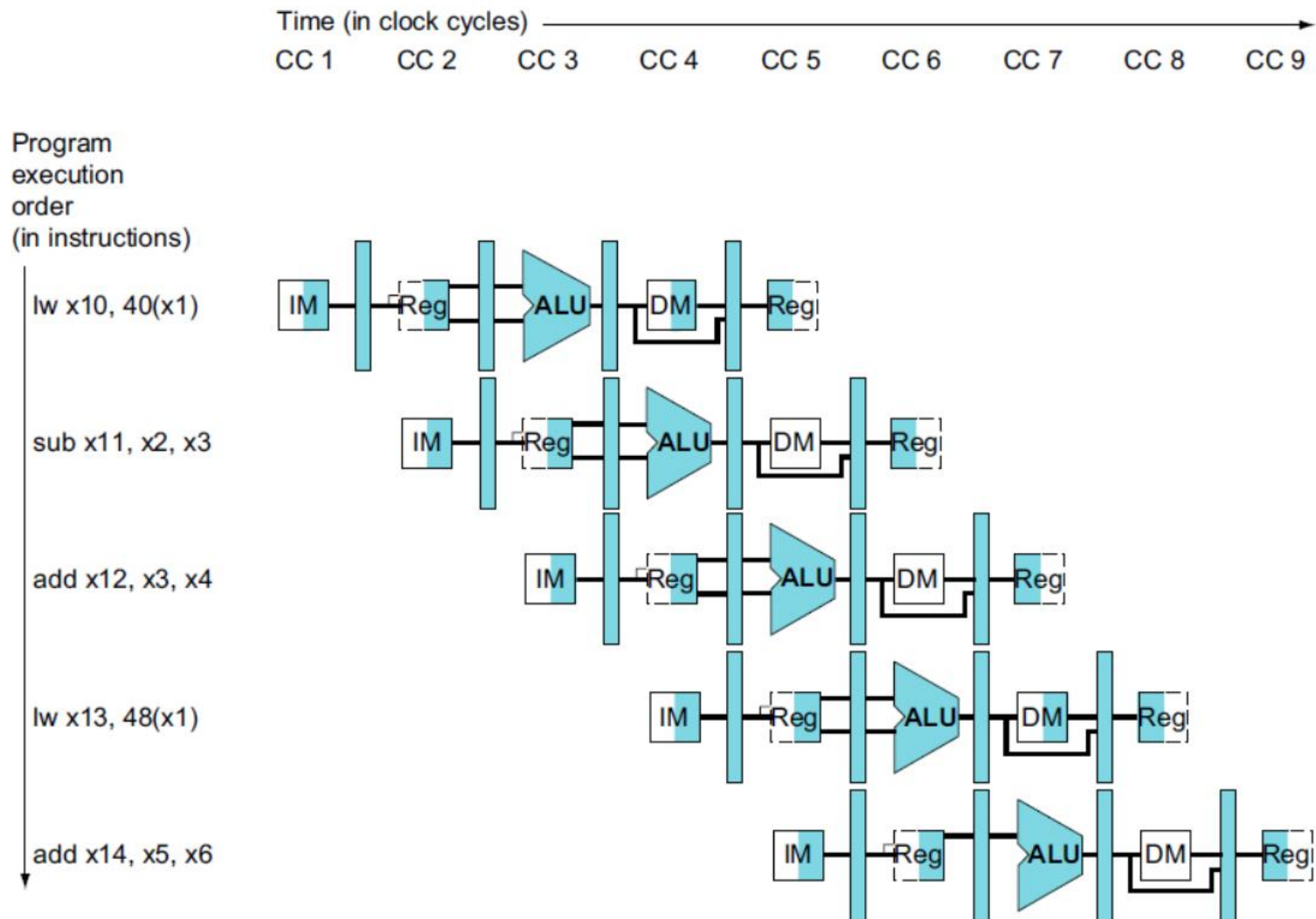


WB for Store

SW
Write-back

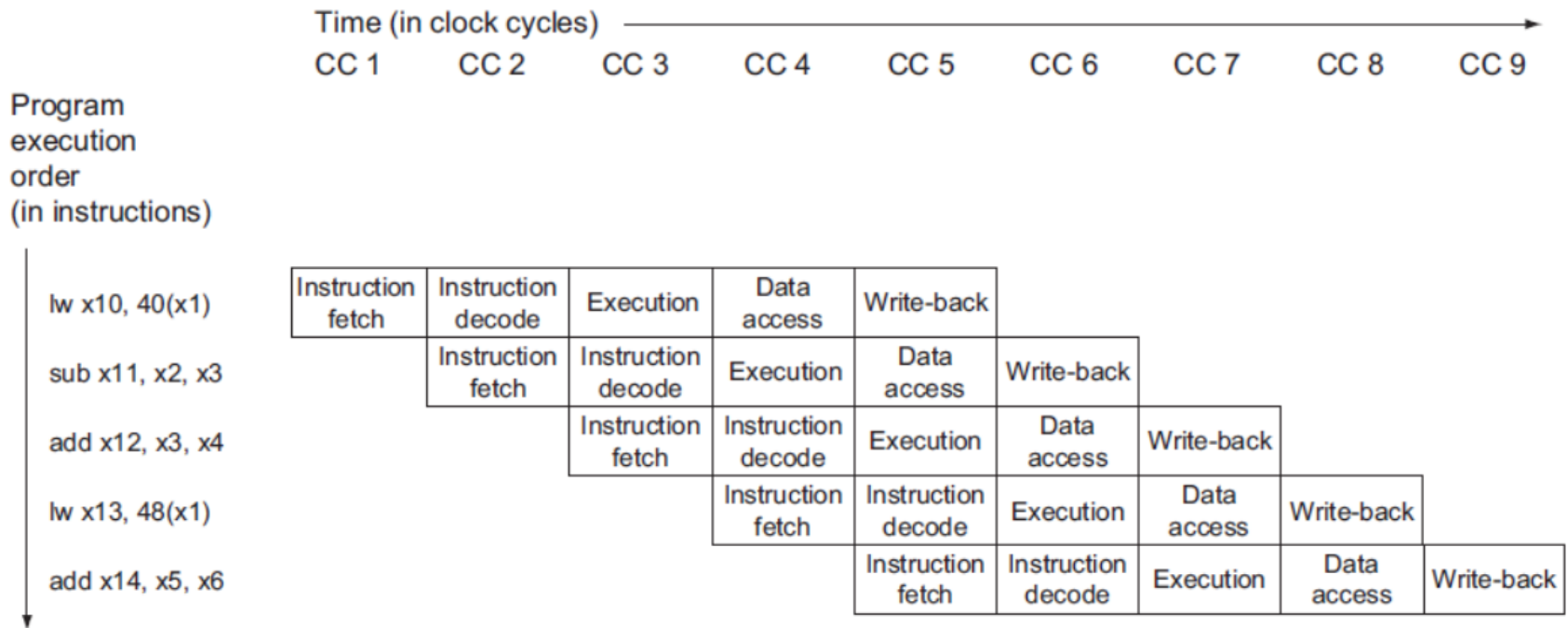


Multi-Cycle Pipeline Diagram



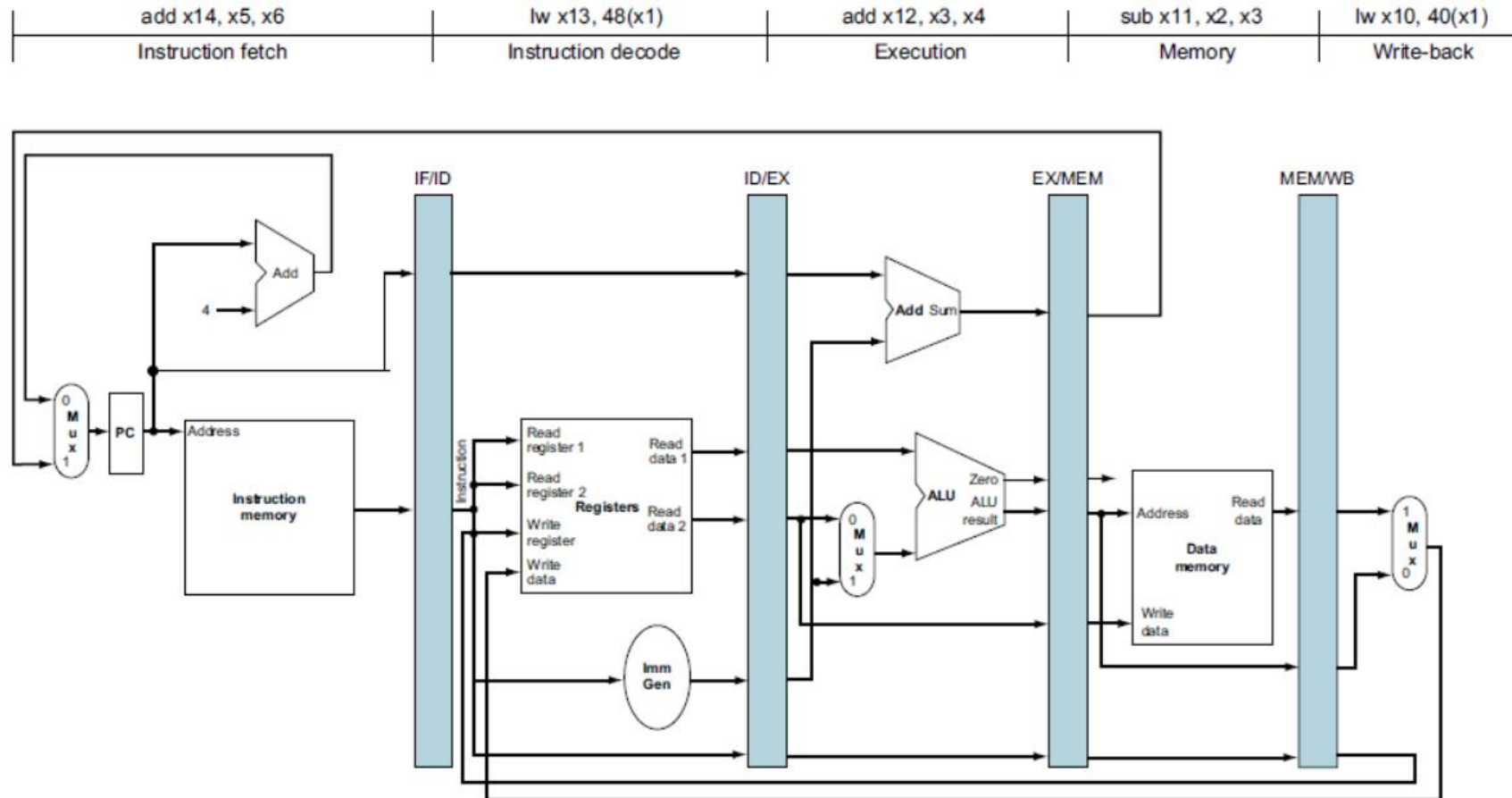
Multi-Cycle Pipeline Diagram

■ Traditional form

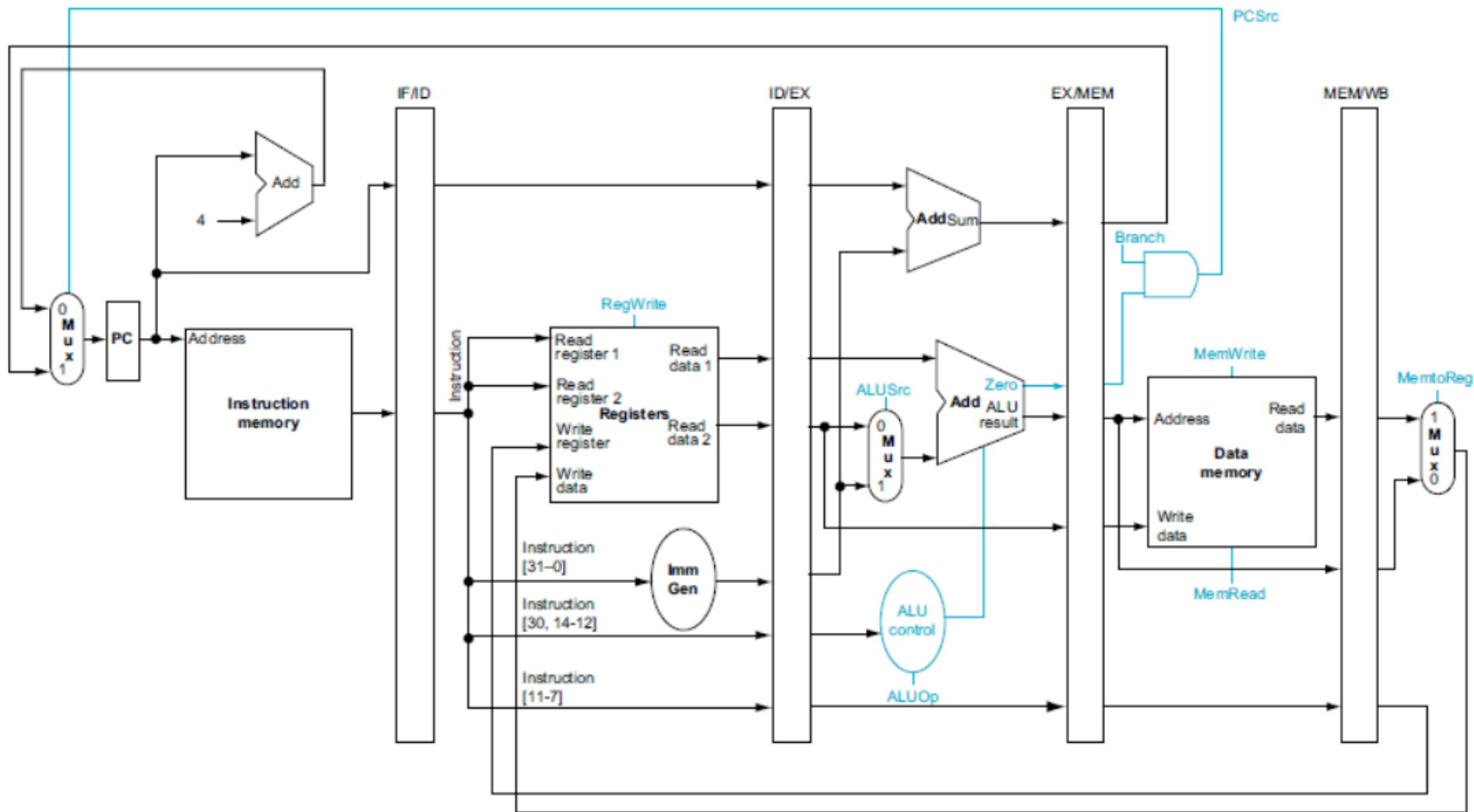


Single-Cycle Pipeline Diagram

- A single-clock-cycle figure is a vertical slice through a multiple-clock-cycle diagram



Pipelined Control (Simplified)



Pipelined Control

- Recall from Single-cycle control

Instruction	ALUOp	operation	Funct7 field	Funct3 field	Desired ALU action	ALU control Input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

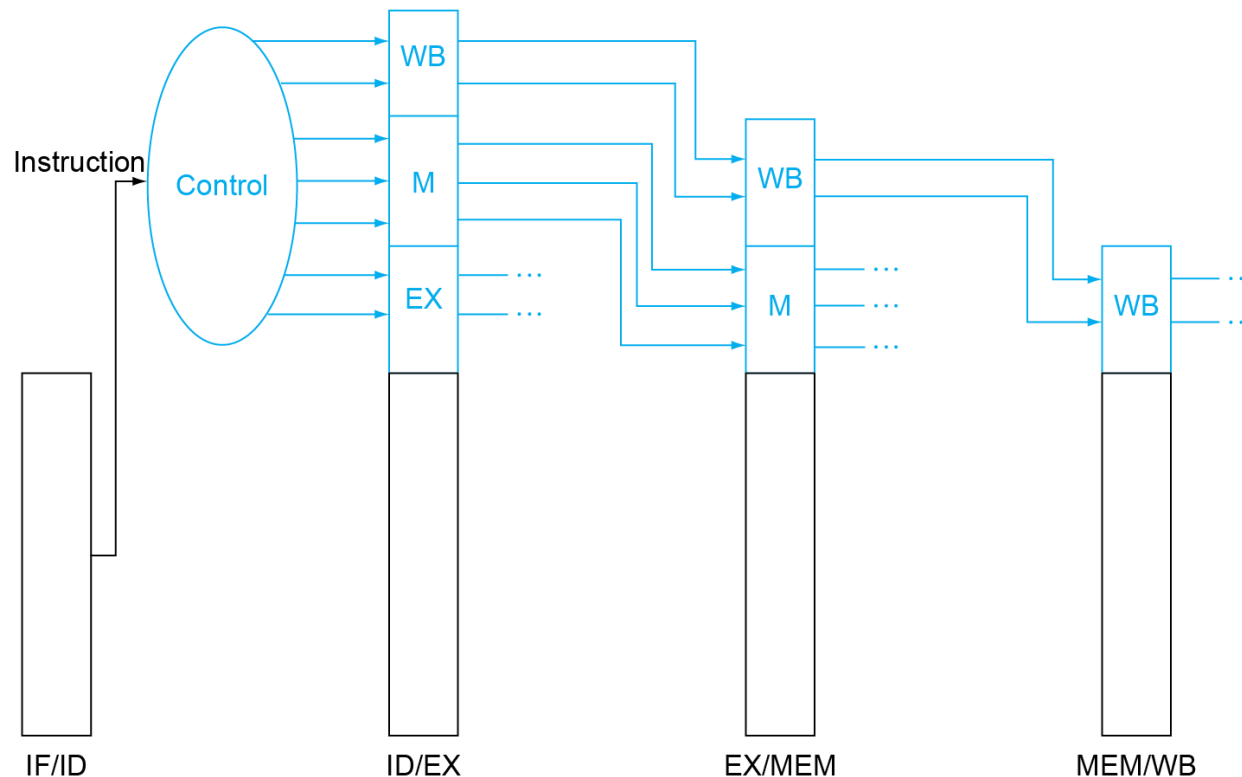
Pipelined Control

- Recall from Single-cycle control

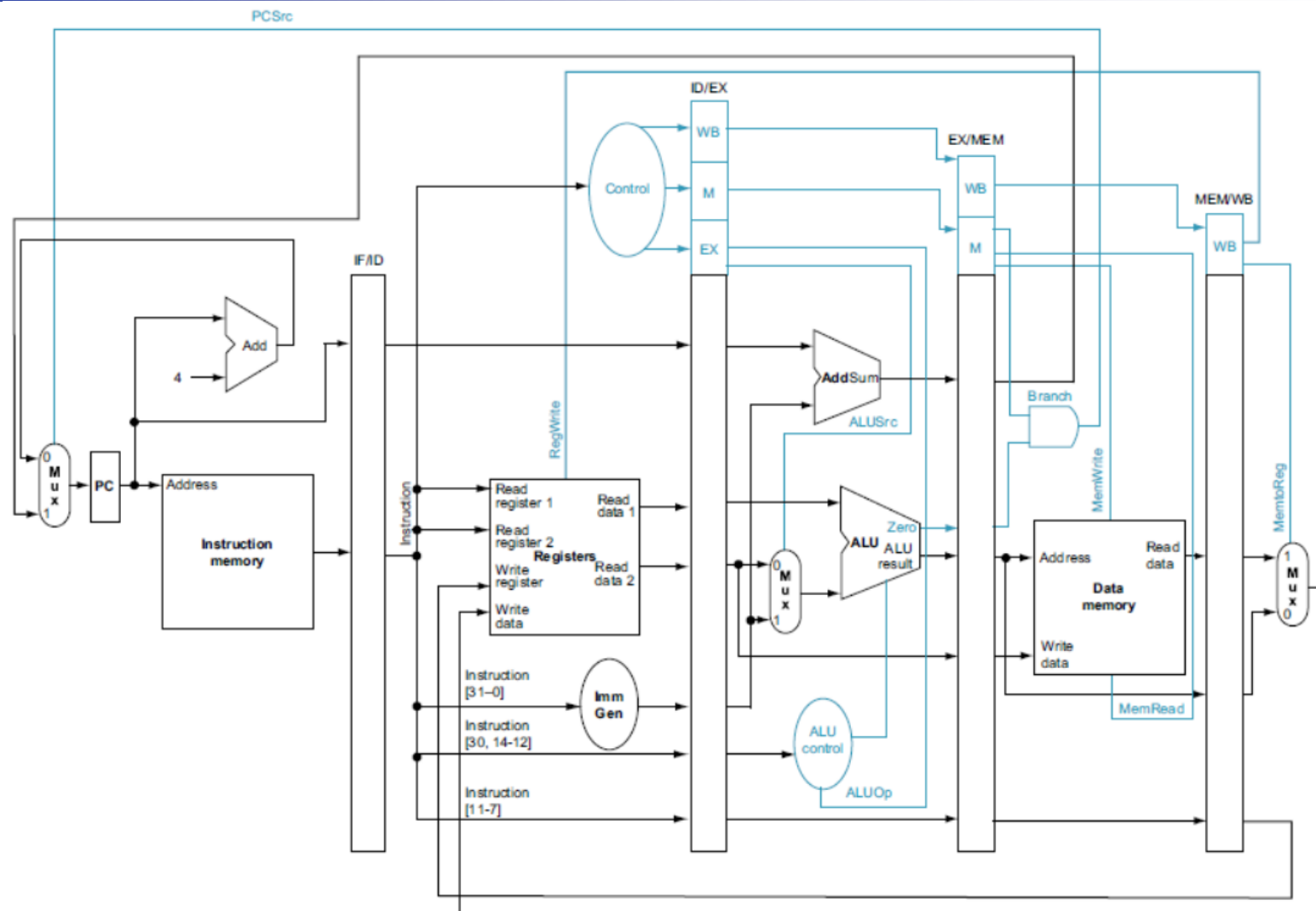
Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

Pipelined Control

- Control signals derived from instruction
 - As in single-cycle implementation



Pipelined Control



Chapter Outline

Chapter 4 – The Processor

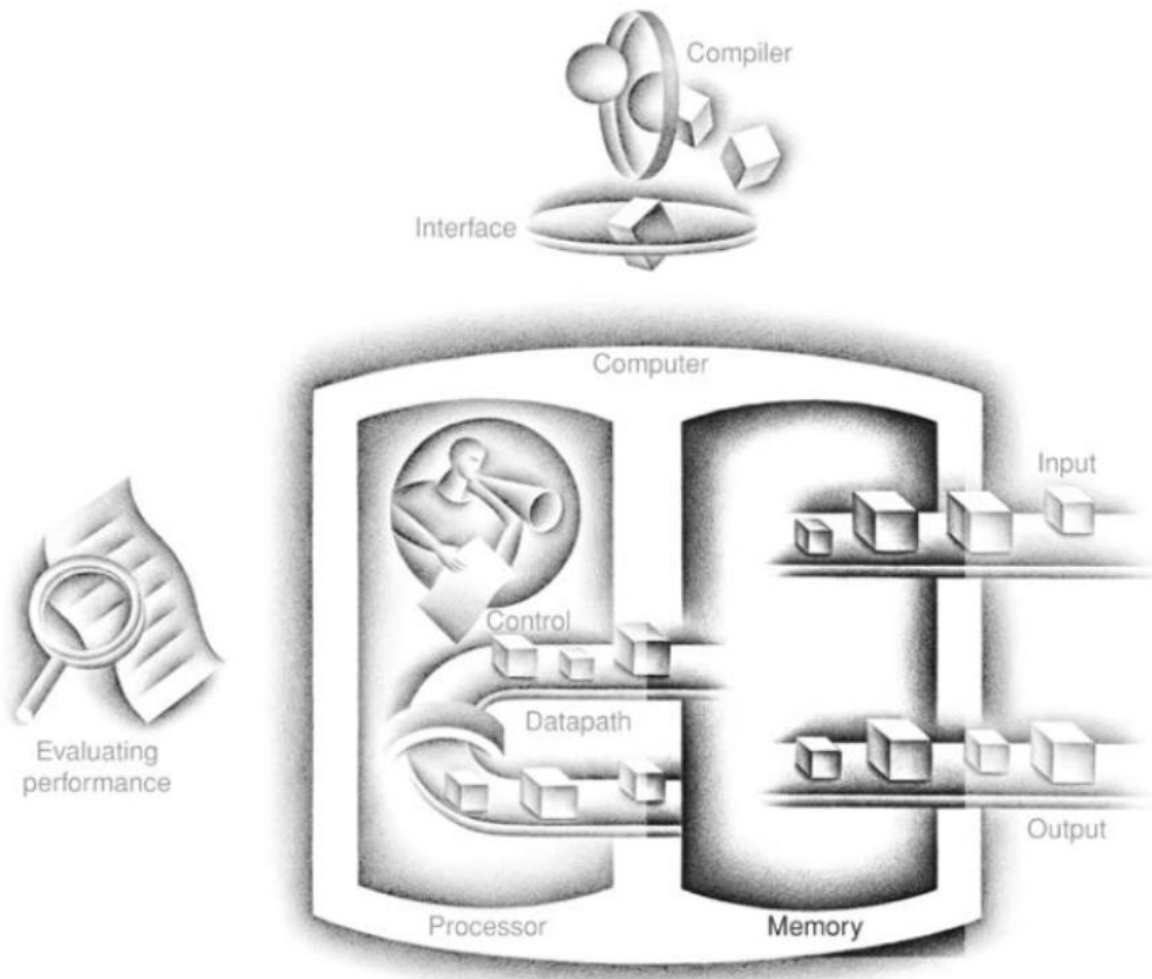
- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Introduction

Ideally one would desire an indefinitely large memory capacity such that any particular ... word would be immediately available. ...

We are ... forced to recognize the possibility of constructing a *hierarchy of memories*, each of which has greater capacity than the preceding but which is less quickly accessible

A. W. Burks, H. H. Goldstine, and J. von Neumann,
*Preliminary Discussion of the Logical Design of an
Electronic Computing Instrument*, 1946

Introduction

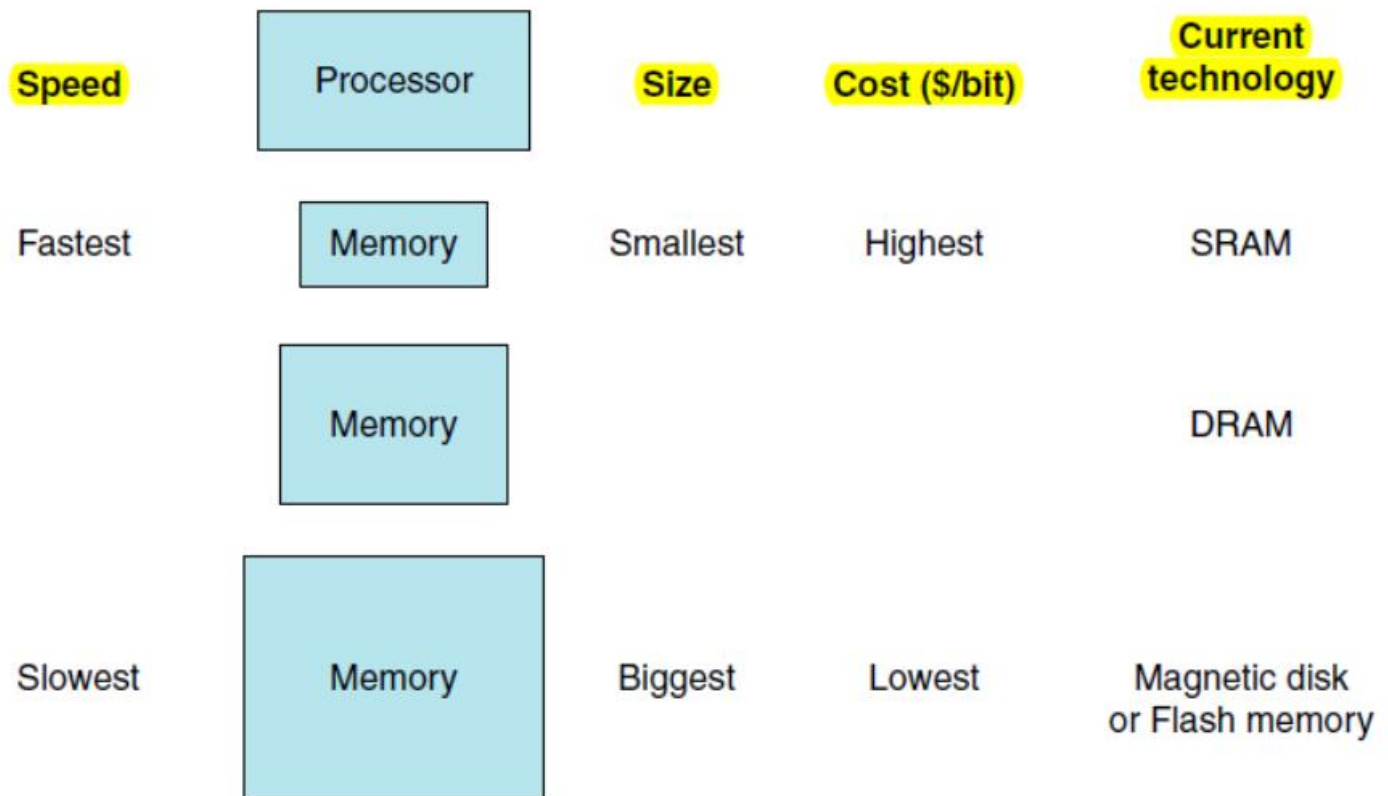
- From the earliest days of computing, programmers have wanted unlimited amounts of fast memory.
- The topics in this chapter aid programmers by creating that illusion
- Analogy of research in library

Principle of Locality

- Programs access a small proportion of their address space at any time
- Temporal locality
 - Items accessed recently are likely to be accessed again soon
 - e.g., instructions in a loop, induction variables
- Spatial locality
 - Example

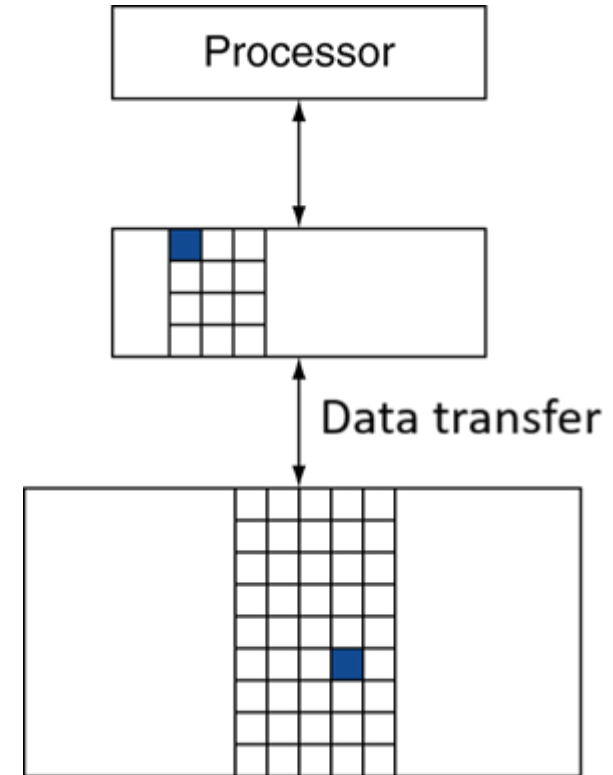
Taking Advantage of Locality

- Memory hierarchy (tradeoff)
- Goal: Maximize use of cheapest memory with access speeds of fastest memory!



Memory Hierarchy Levels

- Block (aka line):
 - Minimum unit of information present at a level / copied from one level to another
- If accessed data is present in upper level
 - Hit: access satisfied by upper level
 - Hit time
 - Hit rate
- If accessed data is absent
 - Miss: block copied from lower level
 - Miss time/Penalty
 - Miss rate



Memory importance

- Concepts used to build memory systems affect many other aspects of a computer, including
 - how applications use the computer
 - how the operating system manages memory
 - how compilers generate code
- Memory is a major factor in determining performance of a program execution on a processor
- Programmers need to understand and exploit the fact that memory is a hierarchy to get good performance
- Similarly, computer designers develop sophisticated mechanisms for improving the memory system performance.
 - Some of these mechanisms we will study in this chapter

Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Memory Technologies

- There are four primary technologies used today in memory hierarchies.
 1. Dynamic RAM (DRAM)
 - Relatively cheaper but slower
 - *Main memory (RAM)* is made using this technology
 2. Static RAM (SRAM)
 - Relatively expensive but faster
 - *Caches* are made using this technology
 3. Magnetic disk
 - Cheapest but slowest
 - *Hard disks* are made using this technology
 4. Flash Disk
 - Often used as portable storage devices e.g *USBs*, SD cards etc.

Memory Technologies

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12
Magnetic disk	5,000,000–20,000,000 ns	\$0.01–\$0.02

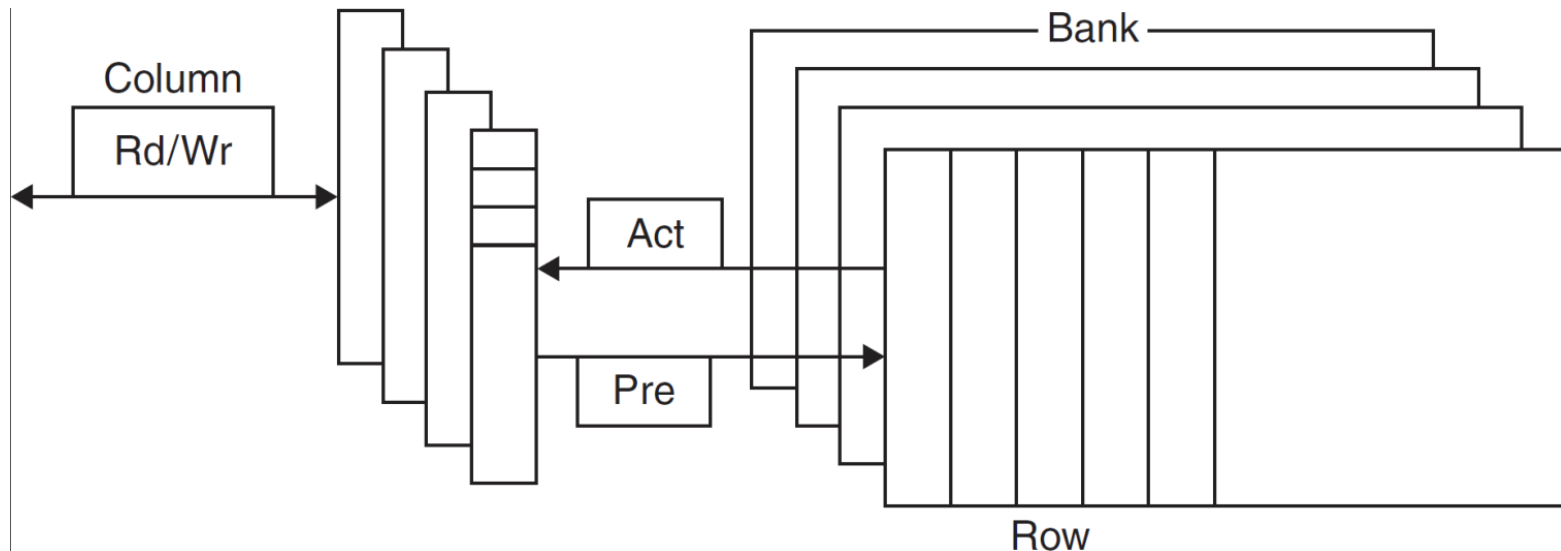
The next few slides will discuss in detail these technologies

SRAM Technology

- SRAMs are integrated circuits working as memory arrays
 - 6-8 transistors latched together to store a bit
- Single access port that can provide either a read or a write
- Fixed access time to any datum, though read and write times can differ
- Does not need refreshing to keep the data unlike DRAMs
- Now all levels of caches are integrated onto the processor chip (thanks to Moore's law)

DRAM Technology

- Data stored as a charge in a capacitor
 - Charge leakages over time and hence must periodically be refreshed, hence the term *dynamic*
- Use a single transistor per bit and hence **denser and cheaper** than SRAMs



DRAM Technology

Moore's
Law at
play here!

Year introduced	Chip size	\$ per GiB	Total access time to a new row/column	Average column access time to existing row
1980	64 Kibibit	\$6,480,000	250 ns	150 ns
1983	256 Kibibit	\$1,980,000	185 ns	100 ns
1985	1 Mebibit	\$720,000	135 ns	40 ns
1989	4 Mebibit	\$128,000	110 ns	40 ns
1992	16 Mebibit	\$30,000	90 ns	30 ns
1996	64 Mebibit	\$9,000	60 ns	12 ns
1998	128 Mebibit	\$900	60 ns	10 ns
2000	256 Mebibit	\$840	55 ns	7 ns
2004	512 Mebibit	\$150	50 ns	5 ns
2007	1 Gibibit	\$40	45 ns	1.25 ns
2010	2 Gibibit	\$13	40 ns	1 ns
2012	4 Gibibit	\$5	35 ns	0.8 ns
2015	8 Gibibit	\$7	30 ns	0.6 ns
2018	16 Gibibit	\$6	25 ns	0.4 ns

Advanced DRAM Features

- Row buffer
 - Allows several words to be read and refreshed in parallel -> Burst mode
- Synchronous DRAM
 - Allows for data transfers at both rising and falling edges of the clock (Double data-rate (DDR))
 - Improves bandwidth
- Dual-inline-Memory Modules (DIMM)
 - For servers which require multiple DRAMs (typically 4-16)
 - Improves bandwidth